

Class 7: Machine Learning 1

Julissa Gonzalez(A18495188)

Table of contents

Background	1
K-means clustering	1
Heirarchial Clustering	11
Hands on with PCA	13
Analysis of UK food data	13
Data Import	13
Tidy the data	17
Create grouped bar plot	17
Exporatory analysis	22

Background

Today we will explore some core machine learning methods that are very popular in bioinformatics. These include **clustering** and **dimensionality reduction**.

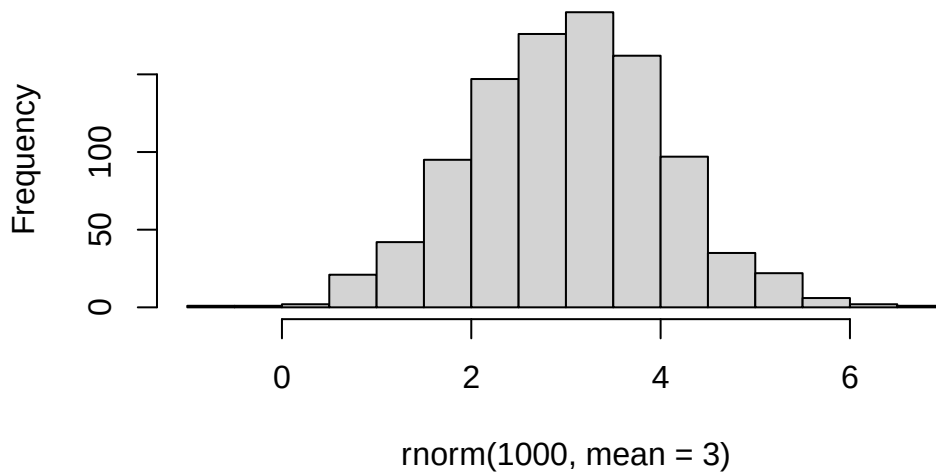
K-means clustering

The main function in “base R for K-means clustering is called `kmeans()`

Before we go too deep, lets make up some “simple” data that we can cluster and know if we are getting a good answer or not. To do this we use the `rnorm()`

```
hist(rnorm(1000,mean=3))
```

Histogram of rnorm(1000, mean = 3)



```
rnorm(30,-3)
```

```
[1] -3.875610 -1.783477 -2.184690 -2.999264 -4.144752 -3.560838 -3.340341  
[8] -2.661777 -2.459793 -2.895749 -3.452803 -1.283614 -3.475207 -1.909313  
[15] -2.561081 -1.782018 -2.113098 -3.023463 -3.206141 -2.248360 -3.358999  
[22] -2.374498 -5.075670 -2.263712 -3.542384 -2.503396 -2.776198 -2.984264  
[29] -2.831289 -2.890178
```

```
rnorm(30,+3)
```

```
[1] 3.0848085 3.3198299 4.2269957 4.1613466 3.0749042 2.9760628 2.3146393  
[8] 4.8282165 3.1836028 3.9099761 1.9731976 3.1687153 3.3011145 3.6349601  
[15] 1.9386088 2.4188625 1.5733656 3.9296900 1.3626855 0.5106606 3.1497830  
[22] 3.1376340 2.9625297 2.9529011 1.5349068 4.4871390 2.7823421 1.7988458  
[29] 4.3570675 2.2592084
```

```
x<-c(rnorm(30,-3), rnorm(30,+3))
```

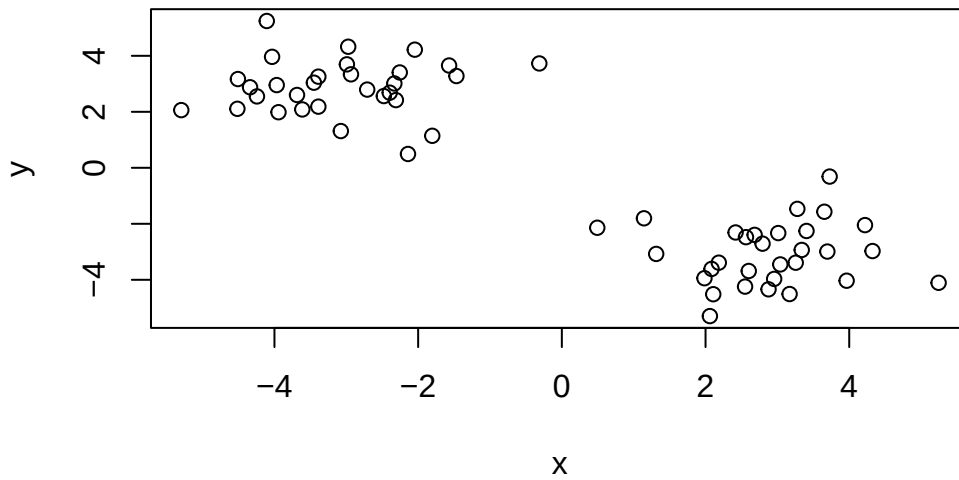
```
z<-cbind(x=x,y=rev(x))
```

```
z
```

	x	y
[1,]	-4.2436652	2.5500279
[2,]	-1.5693333	3.6537387
[3,]	-3.6128108	2.0840555
[4,]	-2.3313892	3.0117649
[5,]	-2.0467654	4.2194927
[6,]	-3.6859836	2.6014608
[7,]	-3.3893686	3.2554725
[8,]	-3.4513558	3.0403652
[9,]	-2.3956016	2.6832640
[10,]	-2.4776760	2.5628241
[11,]	-2.9348895	3.3391115
[12,]	-4.1069374	5.2441303
[13,]	-2.2560248	3.4052224
[14,]	-4.5160877	2.1075272
[15,]	-2.7092162	2.7938242
[16,]	-3.0762591	1.3132445
[17,]	-5.2970278	2.0606907
[18,]	-1.4671782	3.2778677
[19,]	-2.9736670	4.3242809
[20,]	-2.9920011	3.6954549
[21,]	-4.3399312	2.8766838
[22,]	-2.1413931	0.4932216
[23,]	-0.3132071	3.7276363
[24,]	-4.5087768	3.1701354
[25,]	-3.9438989	1.9848423
[26,]	-3.3887055	2.1845327
[27,]	-4.0331169	3.9639866
[28,]	-3.9686209	2.9552310
[29,]	-2.3099113	2.4183461
[30,]	-1.8037795	1.1431175
[31,]	1.1431175	-1.8037795
[32,]	2.4183461	-2.3099113
[33,]	2.9552310	-3.9686209
[34,]	3.9639866	-4.0331169
[35,]	2.1845327	-3.3887055
[36,]	1.9848423	-3.9438989
[37,]	3.1701354	-4.5087768
[38,]	3.7276363	-0.3132071
[39,]	0.4932216	-2.1413931
[40,]	2.8766838	-4.3399312
[41,]	3.6954549	-2.9920011
[42,]	4.3242809	-2.9736670

```
[43,] 3.2778677 -1.4671782
[44,] 2.0606907 -5.2970278
[45,] 1.3132445 -3.0762591
[46,] 2.7938242 -2.7092162
[47,] 2.1075272 -4.5160877
[48,] 3.4052224 -2.2560248
[49,] 5.2441303 -4.1069374
[50,] 3.3391115 -2.9348895
[51,] 2.5628241 -2.4776760
[52,] 2.6832640 -2.3956016
[53,] 3.0403652 -3.4513558
[54,] 3.2554725 -3.3893686
[55,] 2.6014608 -3.6859836
[56,] 4.2194927 -2.0467654
[57,] 3.0117649 -2.3313892
[58,] 2.0840555 -3.6128108
[59,] 3.6537387 -1.5693333
[60,] 2.5500279 -4.2436652
```

```
plot(z)
```



p<-1:5

```
cbind(p,p)
```

```
      p p
[1,] 1 1
[2,] 2 2
[3,] 3 3
[4,] 4 4
[5,] 5 5
```

```
rbind(p,p)
```

	[,1]	[,2]	[,3]	[,4]	[,5]
p	1	2	3	4	5
p	1	2	3	4	5

rev(p)

[1] 5 4 3 2 1

Now we can `runkmeans()` on this input `z` and see what the results look like.

```
km<- kmeans(z, centers=2)
km
```

K-means clustering with 2 clusters of sizes 30, 30

Cluster means:

	x	y
1	-3.076153	2.871385
2	2.871385	-3.076153

Clustering vector:

[illegible]

Within cluster sum of squares by cluster:

```
[1] 63.8401 63.8401
```

```
(between_SS / total_SS = 89.3 %)
```

Available components:

```
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"
```

```
attributes(km)
```

```
$names
```

```
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"
```

```
$class
```

```
[1] "kmeans"
```

Q. How many points are in each cluster?

```
km$size
```

```
[1] 30 30
```

Q. What “components of your result object details cluster assignment/membership?”

```
km$cluster
```

```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
[39] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
```

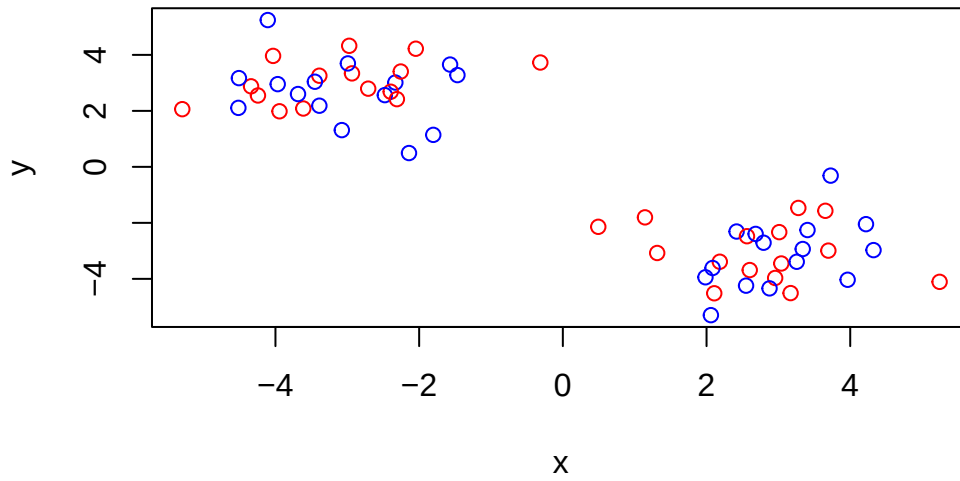
Q. What “components of your result object details cluster center

```
km$centers
```

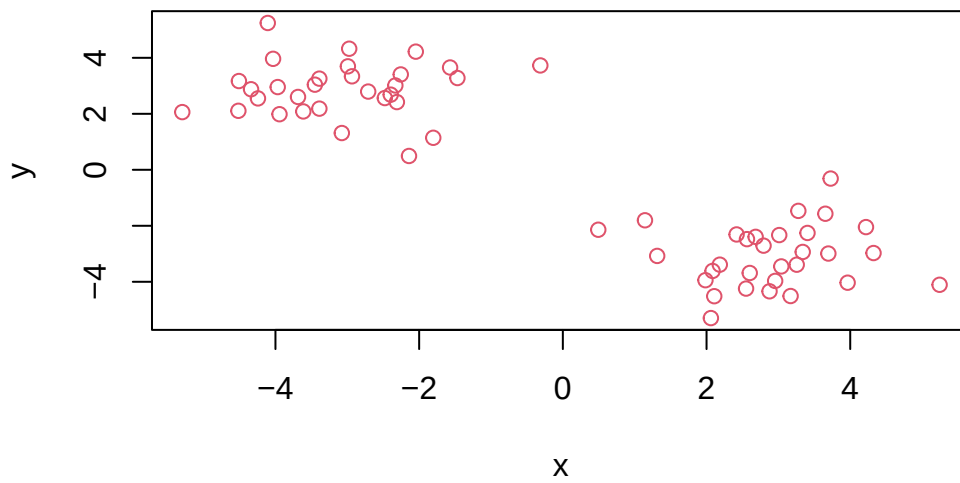
```
      x      y
1 -3.076153 2.871385
2  2.871385 -3.076153
```

Q. Plot z colored by the kmeans cluster assignment and add cluster centers as blue points

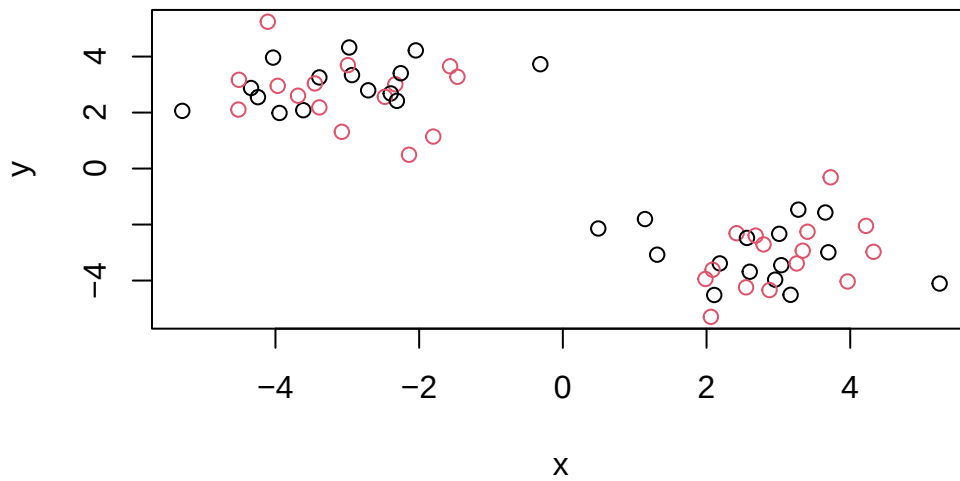
```
plot(z, col=c("red","blue"))
```



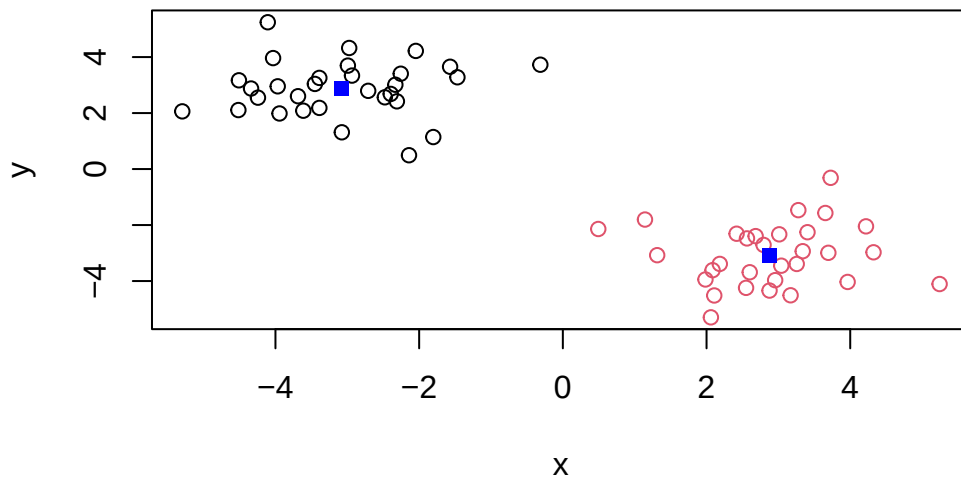
```
plot(z, col=2)
```



```
plot(z, col=c(1,2))
```

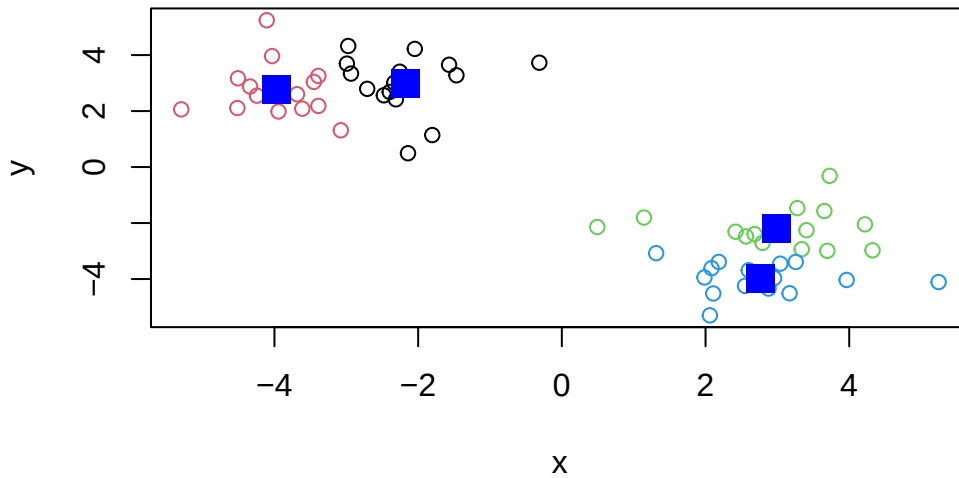


```
plot(z, col=km$cluster)  
points (km$centers, col="blue", pch=15)
```

Q. Run a K-means clustering and plot the results asking for 4 clusters (K=4)

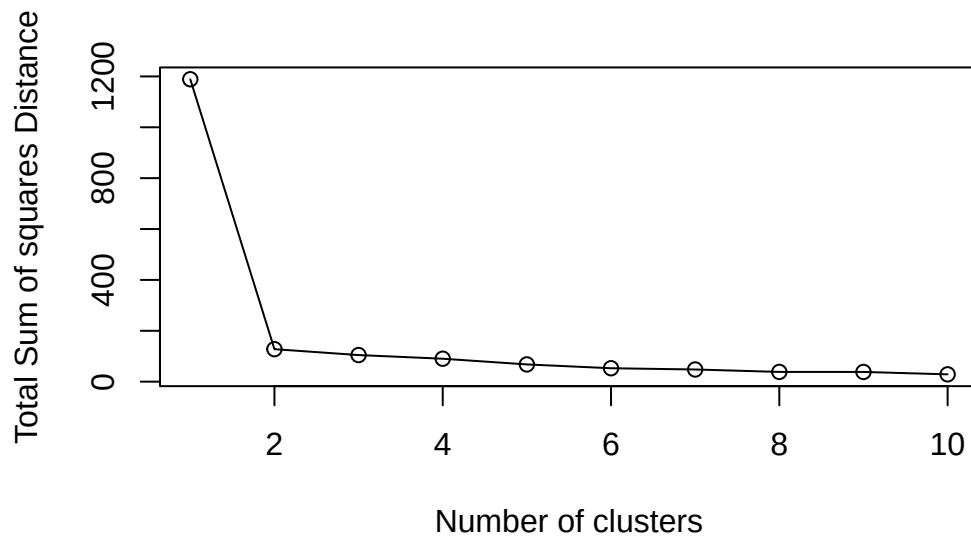
```
km4<- kmeans(z,centers=4)
plot(z, col=km4$cluster)
points(km4$centers, col="blue",pch=15,cex=2 )
```



N.B. You need to tell K-means the number of clusters(i.e. set `center=2`)!!

one approach is to try different valurs for `centers` and then pick the best...

```
ans<- NULL
for(i in 1:10){
  km<- kmeans(z, centers=i)
  ans<-c(ans,km$tot.withinss)
}
plot(ans,type="o",
      xlab="Number of clusters",
      ylab="Total Sum of squares Distance")
```



Heirarchial Clustering

The main function in “base: R Heirachial Clusrering is called `hclust()`

This function does not take your “raw” data for clustering. You must first build a “distance matrix” from your data and pass this as input to `hclust()`

```
d<-dist(z)
hc<-hclust(d)
hc
```

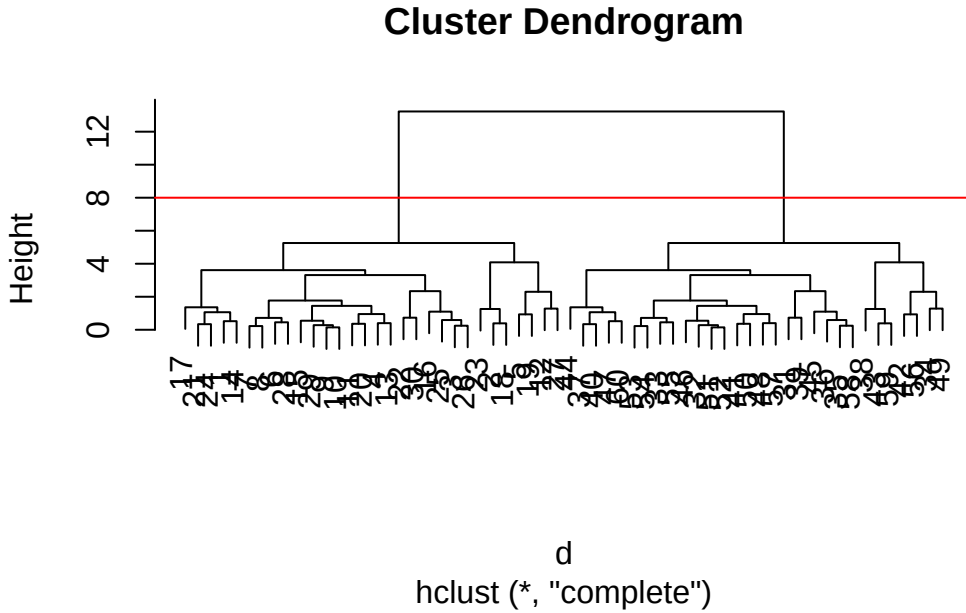
Call:

```
hclust(d = d)
```

```
Cluster method   : complete
Distance         : euclidean
Number of objects: 60
```

There is a bespoke `plot()` method for `hclust()` result objects.

```
plot(hc)
abline(h=8, col="red")
```



Once we have our `hclust` object(our “tree” of “cluster dendrogram”) we can “*cut*” the tree to reveal the clustering pattern

```
cutree(hc, h=8)
```

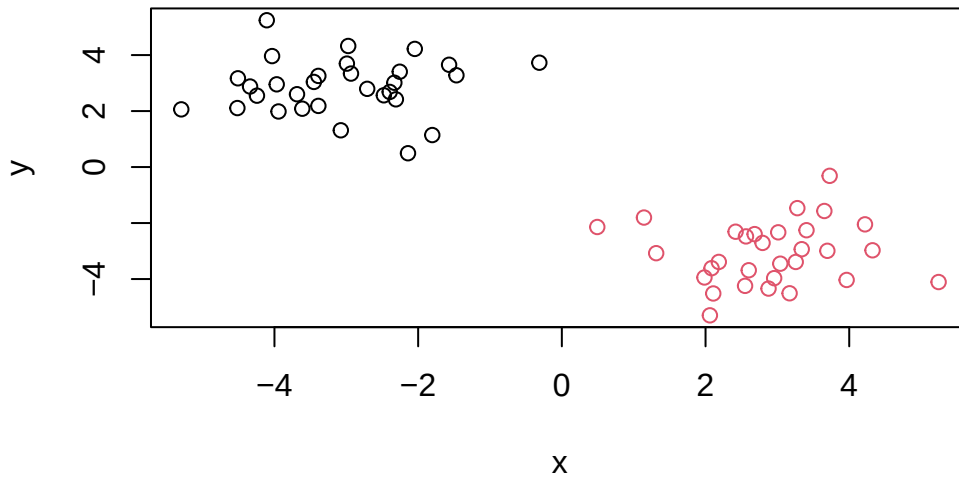
[illegible]

```
cutree(hc, k=4)
```

[1] 1 2 1 1 2 1 1 1 1 1 2 1 1 1 1 2 2 1 1 1 2 1 1 1 2 1 1 3 3 3 4 3 3 3 4
[39] 3 3 3 4 4 3 3 3 3 3 4 3 3 3 3 3 4 3 3 4 3

Q. Make a plot of $\mathbf{z}$ with your hclust results (i.e. colored by cluster membership)

```
grps<- cutree(hc, k=2)
plot(z, col=grps)
```



Hands on with PCA

Principal Component Analysis(PCA) PCA is a dimensionality reduction method that is popular for revealing patterns in complex data sets

Analysis of UK food data

Let's look at some data on eating habits of folks from the UK to see if there are patterns and trends that have some regions being distinct from others.

Data Import

The data is made available in CSV format so we can use the `read.csv()`

```
url<- "https://tinyurl.com/UK-foods"  
x <- read.csv(url)  
dim(x)
```

```
[1] 17  5
```

```
head(x)
```

	X	England	Wales	Scotland	N.Ireland
1	Cheese	105	103	103	66
2	Carcass_meat	245	227	242	267
3	Other_meat	685	803	750	586
4	Fish	147	160	122	93
5	Fats_and_oils	193	235	184	209
6	Sugars	156	175	147	139

Q. How many rows and columns are in your new data frame named x? What R functions could you use to answer this questions?

```
dim(x)
```

```
[1] 17 5
```

```
x <- read.csv(url, row.names=1)
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

Q2. Which approach to solving the ‘row-names problem’ mentioned above do you prefer and why? Is one approach more robust than another under certain circumstances?

I prefer using `read.csv (url, row.names = 1)` because it tells R that the first column should be used as row names. It’s cleaner and less error prone than importing the column first and then manually removing it.

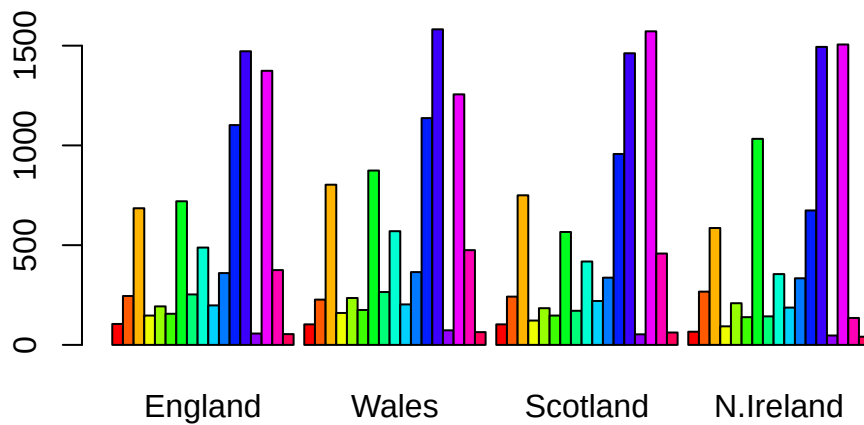
```
# Note how the minus indexing works
rownames(x) <- x[,1]
x <- x[,-1]
head(x)
```

	Wales	Scotland	N.Ireland
105	103	103	66
245	227	242	267
685	803	750	586
147	160	122	93
193	235	184	209
156	175	147	139

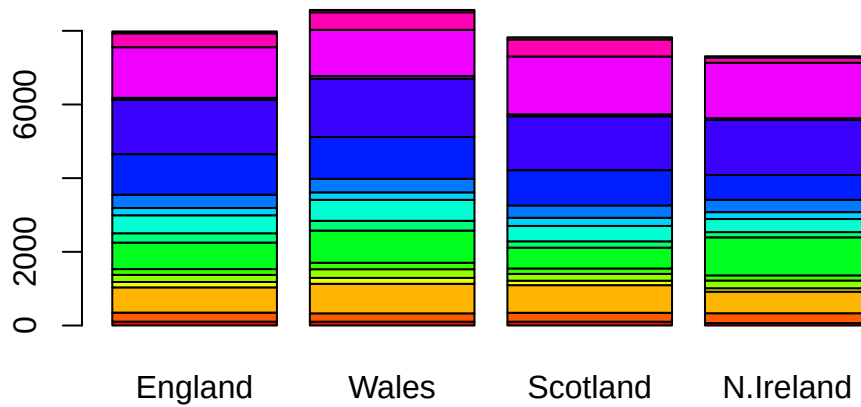
```
x <- read.csv(url, row.names=1)
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

```
barplot(as.matrix(x), beside=T, col=rainbow(nrow(x)))
```



```
barplot(as.matrix(x), beside=F, col=rainbow(nrow(x)))
```



Changing `beside = TRUE` to `beside = FALSE` makes the stacked barplot.

```
dim(x)
```

```
[1] 17  4
```

```
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

Tidy the data

Fix anything that went wrong with data import.

```
library(tidyr)

# Convert data to long format for ggplot with `pivot_longer()`
x_long <- x |>
  tibble::rownames_to_column("Food") |>
  pivot_longer(cols = -Food,
               names_to = "Country",
               values_to = "Consumption")

dim(x_long)
```

```
[1] 68  3
```

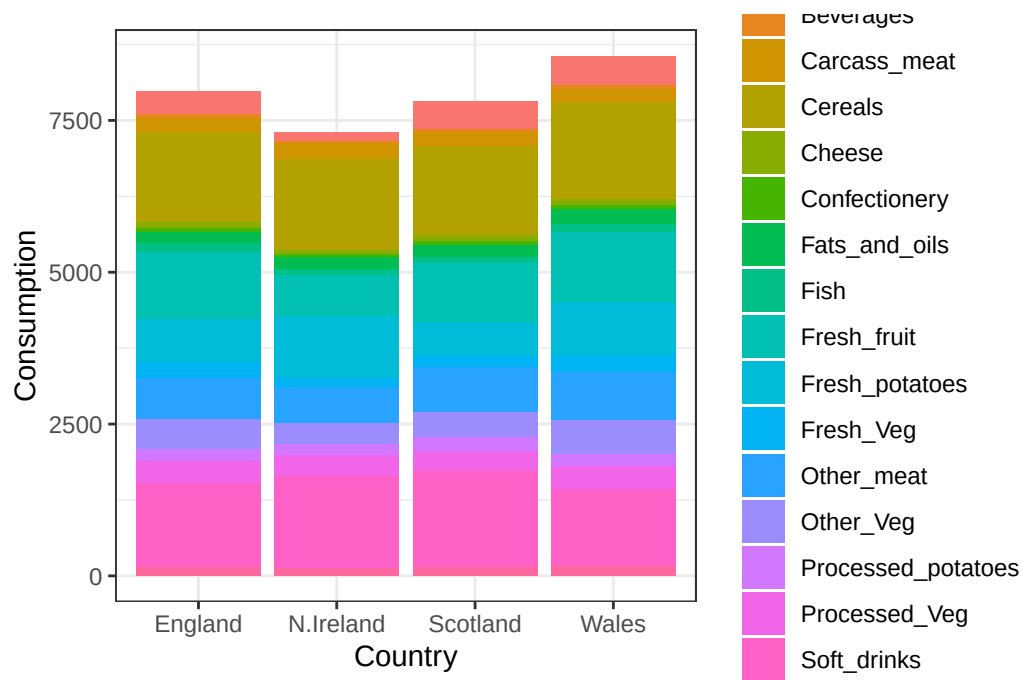
```
head(x_long)
```

```
# A tibble: 6 x 3
  Food          Country Consumption
<chr>         <chr>         <int>
1 "Cheese"      England          105
2 "Cheese"      Wales            103
3 "Cheese"      Scotland         103
4 "Cheese"      N.Ireland         66
5 "Carcass_meat " England          245
6 "Carcass_meat " Wales            227
```

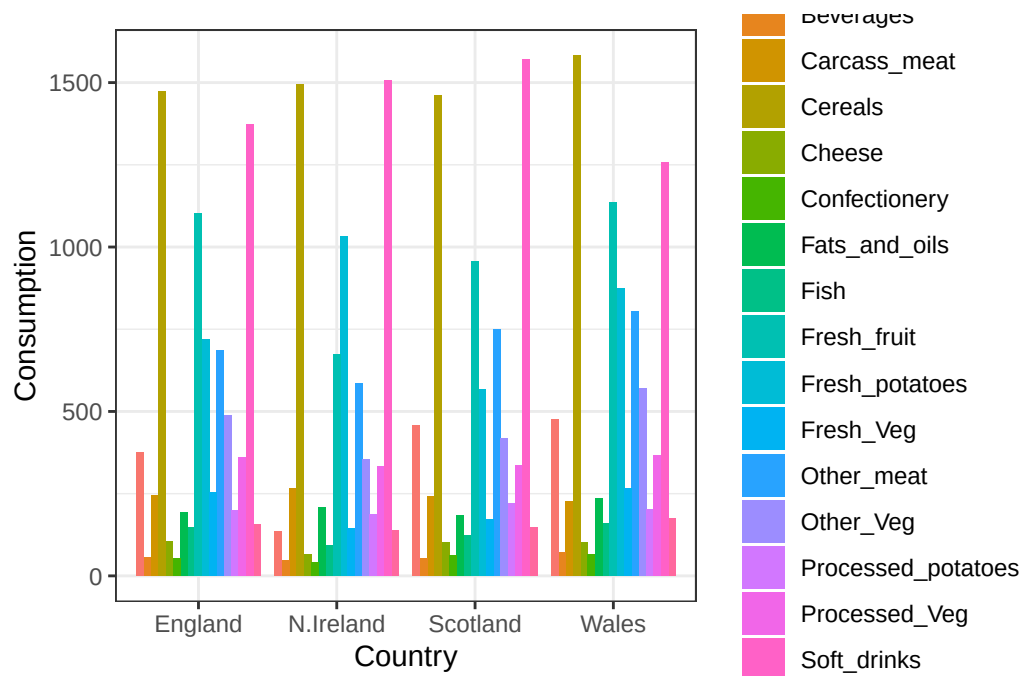
Create grouped bar plot

```
library(ggplot2)
```

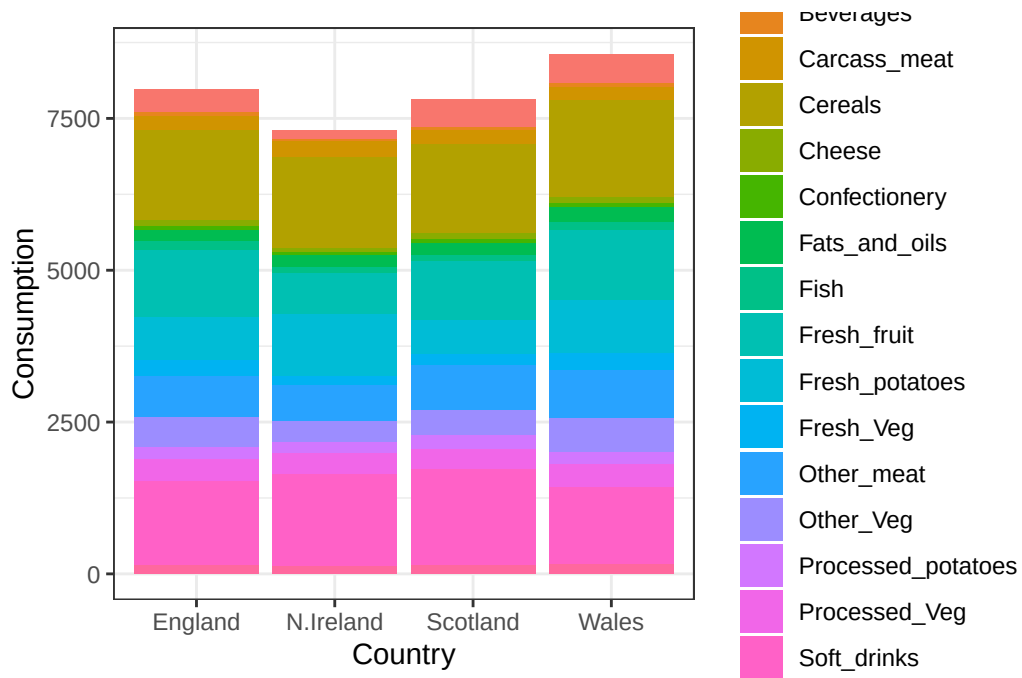
```
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col() +
  theme_bw()
```



```
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "dodge") +
  theme_bw()
```



```
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "stack") +
  theme_bw()
```

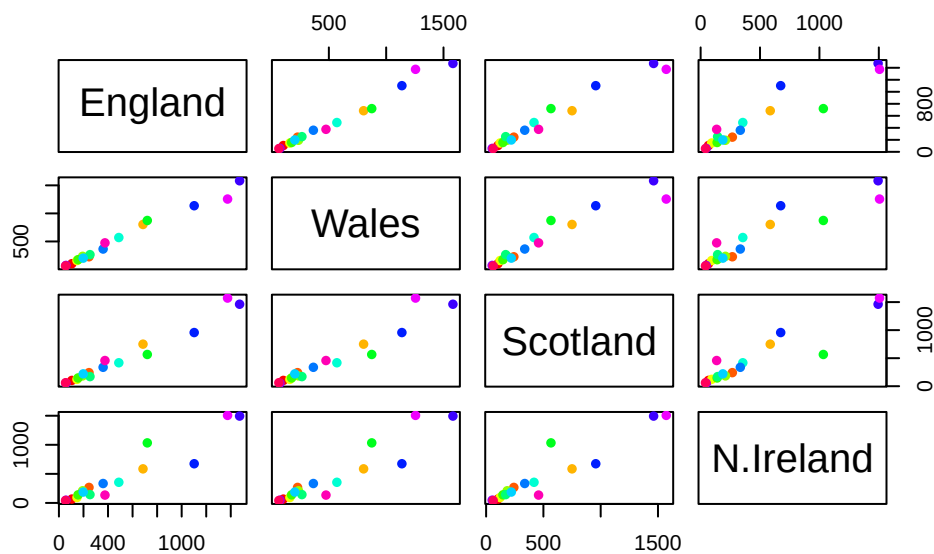


Changing `geom_col(position = "dodge")` to `geom_col(position = "stack")`, or just `geom_col()`, makes the stacked barplot.

. Q5: We can use the `pairs()` function to generate all pairwise plots for our countries. Can you make sense of the following code and resulting figure? What does it mean if a given point lies on the diagonal for a given plot?

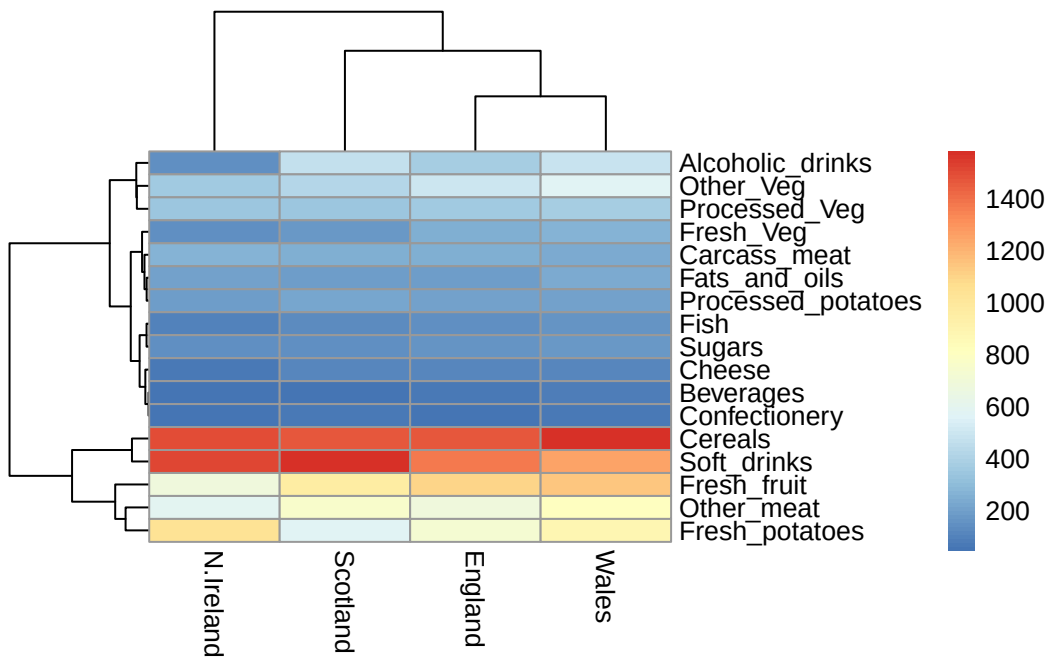
A point on the diagonal means that the two countries have similar consumption values for that food category. The pairs plot compares each country against every other country to show similarities and differences in food consumption patterns.

```
pairs(x, col=rainbow(nrow(x)), pch=16)
```



```
library(pheatmap)

pheatmap( as.matrix(x) )
```



Exporatory analysis

Make some plots to help make sense of obvious trends...

Q6. Based on the pairs and heatmap figures, which countries cluster together and what does this suggest about their food consumption patterns? Can you easily tell what the main differences between N. Ireland and the other countries of the UK in terms of this data-set?

England, Wales, and Scotland cluster more together, meaning they have more similar food consumption patterns. Northern Ireland looks more different from the others, but it is hard to tell exactly which foods drive the difference from only the pairs plot and heatmap.

##PCA

```
pca <- prcomp( t(x) )  
summary(pca)
```

Importance of components:

	PC1	PC2	PC3	PC4
Standard deviation	324.1502	212.7478	73.87622	2.921e-14
Proportion of Variance	0.6744	0.2905	0.03503	0.000e+00
Cumulative Proportion	0.6744	0.9650	1.00000	1.000e+00

The returned `pca` object has components that we can use to make our result figures:

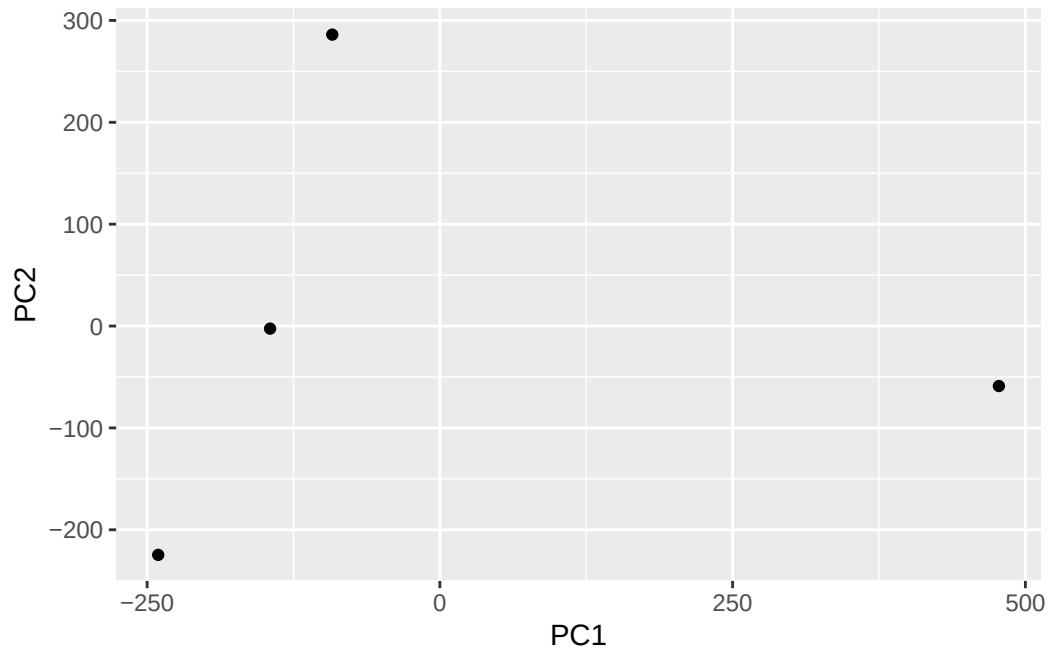
```
attributes(pca)
```

```
$names  
[1] "sdev"      "rotation" "center"   "scale"    "x"
```

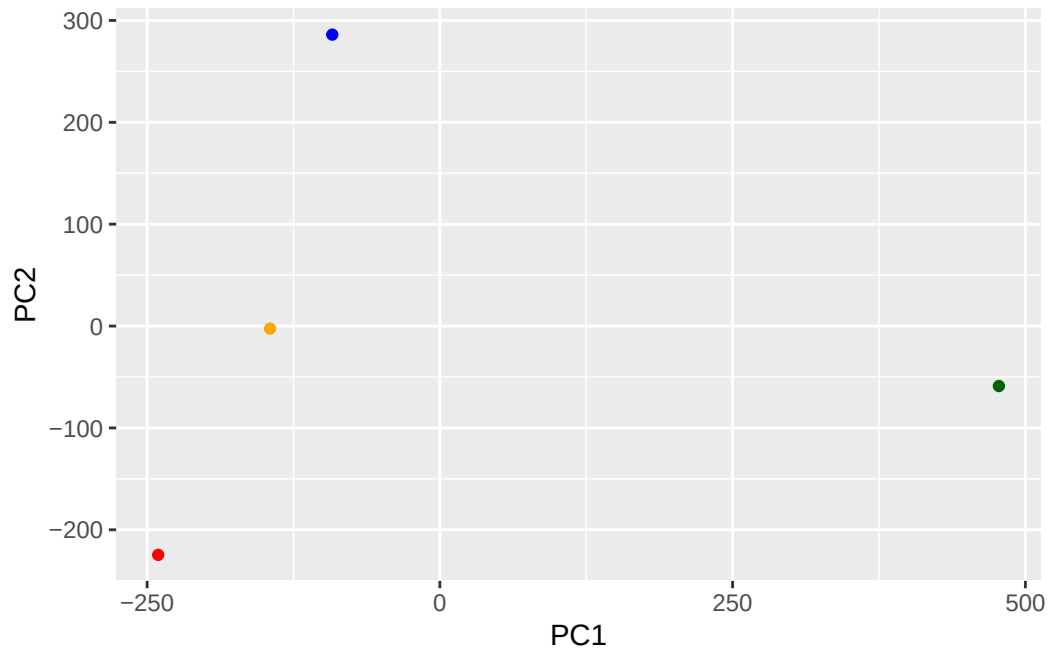
```
$class  
[1] "prcomp"
```

the main result figure from this analysis is called a “PC score plot” or ” orientation ploy” “PC plot” or “PC1 vs PC2 plot”.

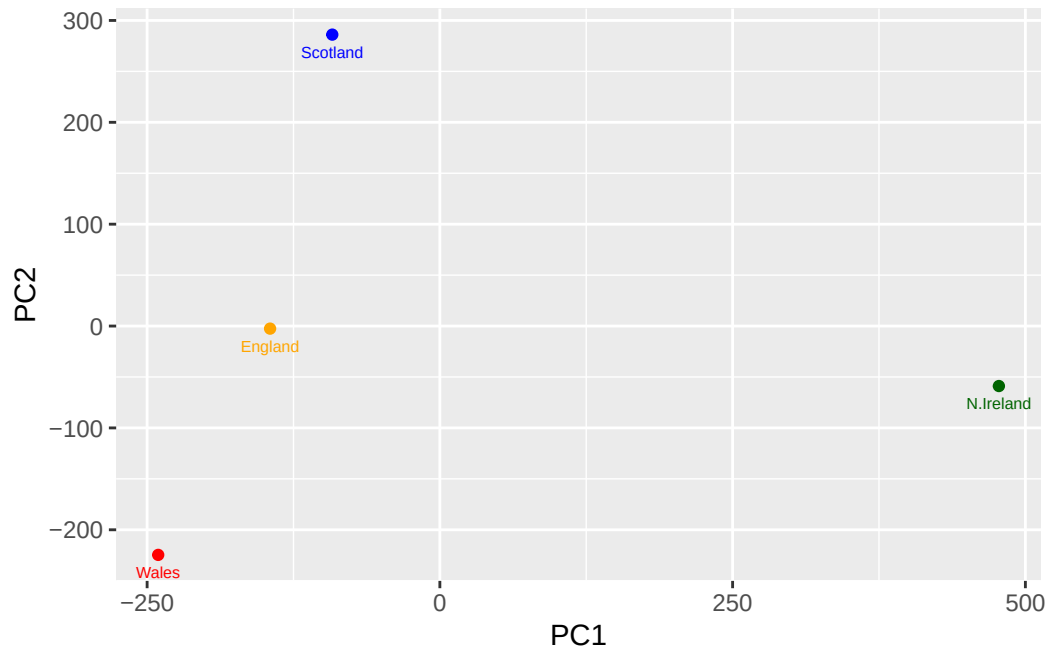
```
library(ggplot2)  
  
ggplot(pca$x)+  
  aes(PC1, PC2)+  
  geom_point()
```



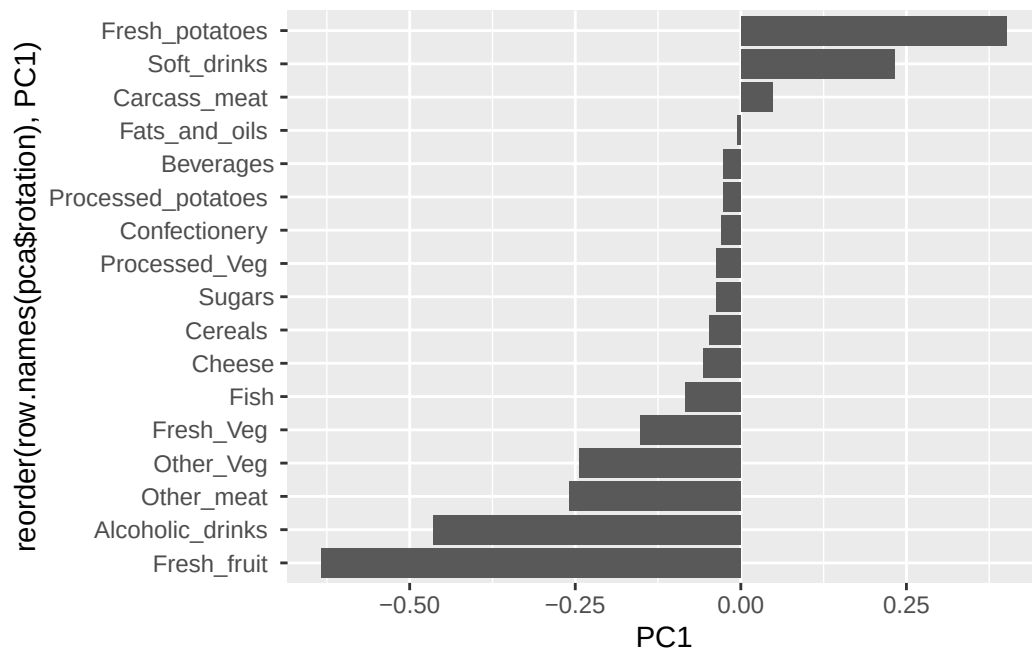
```
mycols<-c("orange","red","blue","darkgreen")
ggplot(pca$x)+
  aes(PC1, PC2)+
  geom_point(col=mycols)
```



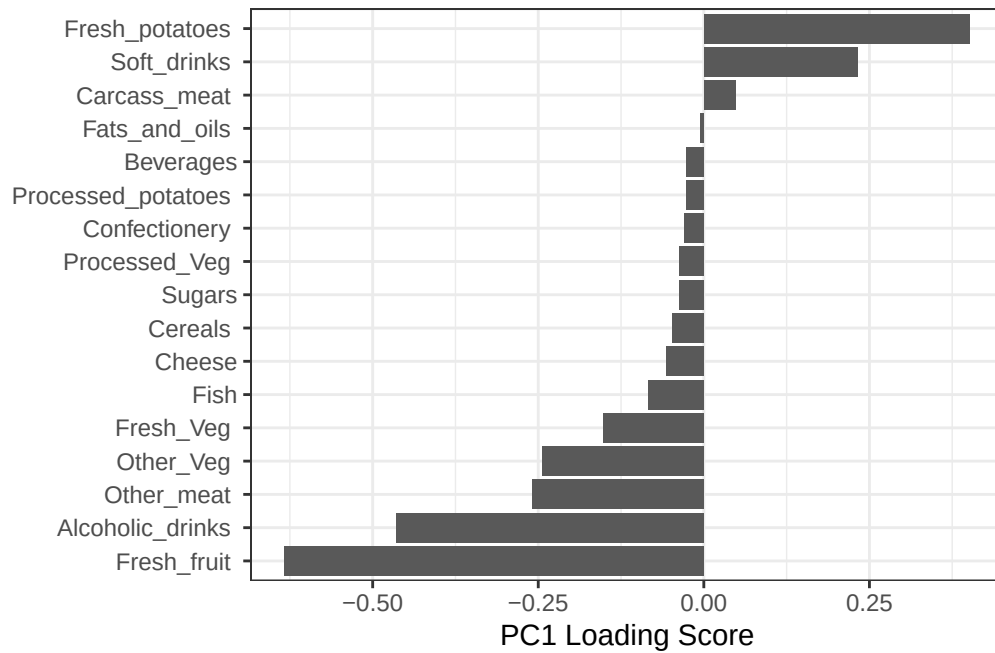
```
ggplot(pca$x)+  
  aes(PC1, PC2,label=row.names(pca$x))+  
  geom_point(col=mycols)+  
  geom_text(size=2, vjust=2, col=mycols)
```

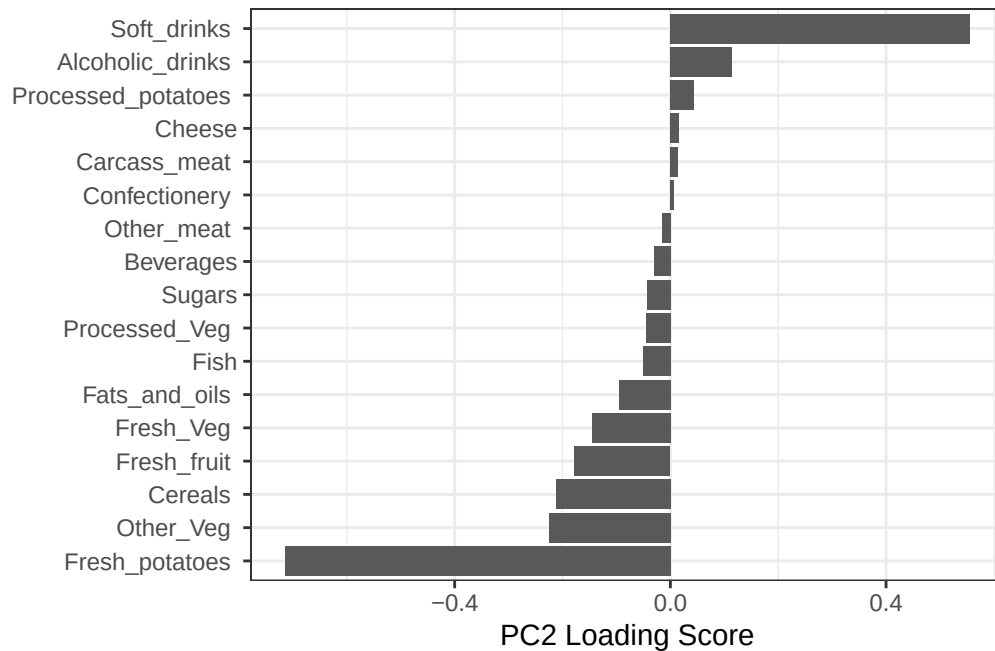
```
ggplot(pca$rotation)+  
  aes(x= PC1,  
      y= reorder(row.names(pca$rotation),PC1))+  
  geom_col()
```



```
ggplot(pca$rotation) +
  aes(x = PC1, y = reorder(row.names(pca$rotation), PC1)) +
  geom_col() +
  xlab("PC1 Loading Score") +
  ylab("") +
  theme_bw()
```



```
ggplot(pca$rotation) +  
  aes(x = PC2, y = reorder(rownames(pca$rotation), PC2)) +  
  geom_col() +  
  xlab("PC2 Loading Score") +  
  ylab("") +  
  theme_bw()
```



PC2 mainly separates the countries based on the foods with the largest positive and negative loading scores. The most important foods are the ones with the longest bars on the PC2 loading plot.

```
url2 <- "https://tinyurl.com/expression-CSV"
rna.data <- read.csv(url2, row.names = 1)

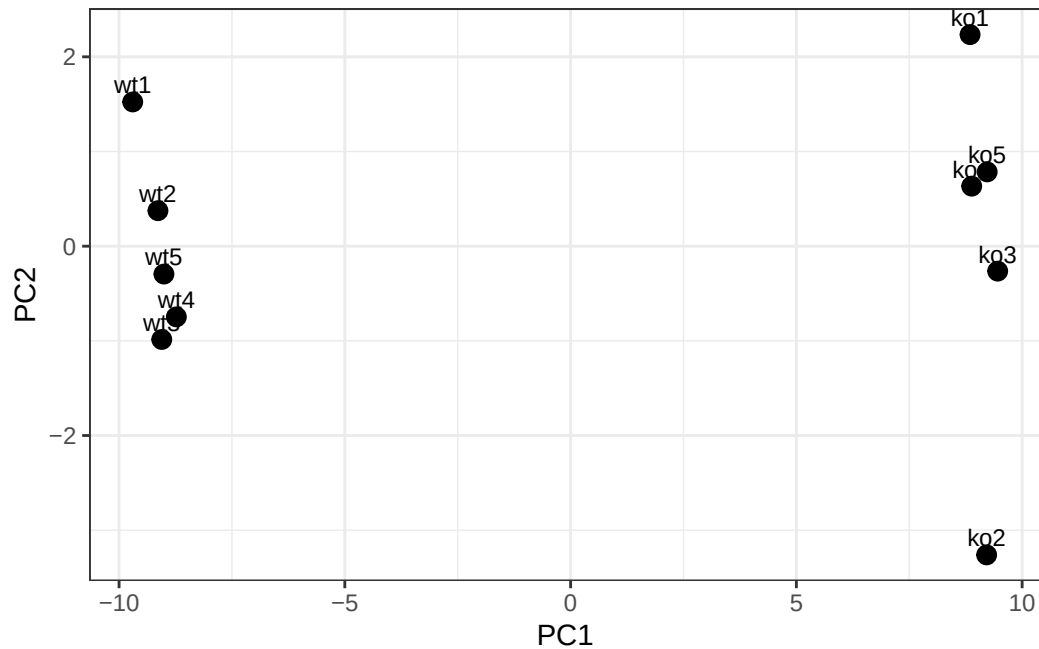
dim(rna.data)
```

```
[1] 100 10
```

```
pca <- prcomp(t(rna.data), scale = TRUE)

df <- as.data.frame(pca$x)
df$Sample <- rownames(df)

ggplot(df) +
  aes(x = PC1, y = PC2, label = Sample) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5, size = 3) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



```
summary(pca)
```

Importance of components:

	PC1	PC2	PC3	PC4	PC5	PC6	PC7
Standard deviation	9.6237	1.5198	1.05787	1.05203	0.88062	0.82545	0.80111
Proportion of Variance	0.9262	0.0231	0.01119	0.01107	0.00775	0.00681	0.00642
Cumulative Proportion	0.9262	0.9493	0.96045	0.97152	0.97928	0.98609	0.99251

	PC8	PC9	PC10
Standard deviation	0.62065	0.60342	3.3e-15
Proportion of Variance	0.00385	0.00364	0.0e+00
Cumulative Proportion	0.99636	1.00000	1.0e+00

There are 100 genes and 10 samples. PC1 gives a useful overview because it explains 92.6% of the variation, and PC1 plus PC2 explain 94.9% total.